

Predicting Ethereum Gas Prices Using Deep Learning: An LSTM-Based Time-Series Forecasting Approach

COM4532 — Blockchain Technologies
Final Project Report

Boran Ömer Doğan (21290594) Arif Erdem Orhan (21290337)
Furkan Çağatay Özbek (21290216)

June 2026

Abstract

Ethereum gas fees are volatile and difficult to anticipate, forcing users either to overpay or to risk transactions stalling. This project formulates next-day gas-price forecasting as a multivariate time-series regression problem and attacks it with a two-layer Long Short-Term Memory (LSTM) network trained on ~ 878 days of post-EIP-1559 on-chain data collected from Etherscan’s public chart exports. Nine input features — six on-chain metrics, an engineered day-over-day price delta, and a cyclically encoded day-of-week signal — are fed to the model through a 30-day sliding window. We compare the LSTM against four baselines (Naïve, 7-day Simple Moving Average, ARIMA, and XGBoost) under a strict chronological train/validation/test split, and we interpret the model with SHAP. On the held-out test set (4 Oct – 30 Dec 2023) the LSTM attains $\text{MAE} = 9.13$, $\text{RMSE} = 11.37$, $\text{MAPE} = 30.2\%$, and $R^2 = 0.458$. The day-of-week feature improves every LSTM metric and SHAP ranks it as the most influential non-price feature. A central, honestly reported finding is that the Naïve “tomorrow = today” baseline remains the strongest predictor ($R^2 = 0.751$): daily average gas price behaves close to a random walk, which is itself the scientifically interesting conclusion of the study.

Contents

1	Introduction and Background	3
1.1	Description of the Problem	3
1.2	Motivation	3
1.3	Current Approaches and Their Limitations	3
1.4	Why Blockchain Technology Is Relevant	4
1.5	Blockchain Ecosystem and Data Source	4
2	Related Work and Technical Background	4
2.1	Blockchain Fundamentals	4
2.2	Smart Contracts, Gas, and DeFi	4
2.3	The Mempool and Fee Market	4
2.4	Blockchain Analytics	5
2.5	Machine Learning Background	5
2.6	Related Studies	5

3	System Design / Problem Construction	6
3.1	Analysis Task	6
3.2	Blockchain Data, Sources, and Entities	6
3.3	Features Extracted	6
3.4	Expected Outputs	6
3.5	End-to-End Pipeline	7
3.6	Design Decisions That Prevent Data Leakage	7
3.7	Rationale for the Window Size and Target	8
3.8	Threats to Validity	8
3.9	Concrete Split	8
4	Implementation	8
4.1	Software Libraries	8
4.2	Data Collection and Merging	9
4.3	Feature Engineering	9
4.4	Preprocessing: Leakage-Free Winsorization, Scaling, and Windowing	9
4.5	LSTM Model and Hyperparameter Search	9
4.6	Walk-Forward Validation	10
4.7	Baselines	10
4.8	Metrics and Explainability	11
5	Evaluation and Results	11
5.1	Evaluation Metrics	11
5.2	Exploratory Data Analysis	11
5.3	Main Results	11
5.4	Effect of the Day-of-Week Feature	12
5.5	Training Behavior	13
5.6	Single-Split vs. Walk-Forward Robustness	13
5.7	Comparison with Related Work	14
5.8	Explainability (SHAP)	14
5.9	Error Analysis	14
6	Discussion and Future Work	15
6.1	Strengths	15
6.2	Limitations	16
6.3	Potential Improvements and Future Work	16
6.4	Conclusion	16
	References	17

1. Introduction and Background

1.1 Description of the Problem

Ethereum is a permissionless, open-source blockchain platform for executing decentralized applications (dApps) and smart contracts. Every operation performed by the Ethereum Virtual Machine is metered in a unit called *gas*, which both rations scarce computational resources and protects the network from spam. Users pay for the gas their transactions consume in Ether (ETH). Because block space is finite while demand fluctuates wildly, the price a user must pay per unit of gas — the *gas price*, quoted in Gwei (10^{-9} ETH) — is highly volatile. During acute demand events such as a high-profile NFT mint or a DeFi liquidation cascade, gas prices can rise by an order of magnitude within hours.

This volatility translates directly into financial uncertainty. A user who sets the gas price too low risks having a transaction sit unconfirmed for hours; a user who sets it too high overpays substantially. The concrete problem addressed by this project is therefore: *given the recent history of on-chain network metrics, can we forecast tomorrow’s average gas price accurately enough to help users and dApp schedulers plan their transactions?*

1.2 Motivation

The economic stakes are large. Aggregated across the network, even small per-transaction inefficiencies amount to enormous wasted expenditure. A reliable forecast would let a patient user defer a non-urgent transaction to a cheaper window, and let an automated scheduler (for example a DeFi rebalancer or an airdrop claimer) submit transactions when fees are low. Unlike a binary “high/low” classifier, a regression model that emits a concrete Gwei estimate can be fed directly into a wallet’s fee field, which is why we adopt a regression formulation.

1.3 Current Approaches and Their Limitations

Existing fee estimators embedded in wallets and explorers (MetaMask, Etherscan’s gas tracker) are overwhelmingly *reactive*: they report a weighted average of recently included gas prices, or an oracle minimum over the last few blocks. These heuristics describe the current state of the network reasonably well but cannot *anticipate* change. They cannot warn a user that fees will spike in ten minutes when an NFT collection opens for minting, nor that waiting an hour will cut fees substantially. What is missing is a *proactive* forecasting system that learns temporal structure from historical data.

EIP-1559 (London hard fork, August 2021) partially smoothed short-term dynamics by replacing the first-price auction with a protocol-set *base fee* that adjusts by at most $\pm 12.5\%$ per block and is *burned* rather than paid to validators, plus a voluntary *priority fee* (tip). The base-fee update rule is

$$\text{BaseFee}_{t+1} = \text{BaseFee}_t \left(1 + 0.125 \cdot \frac{\text{GasUsed}_t - \text{GasTarget}_t}{\text{GasTarget}_t} \right), \quad \text{GasTarget} = \frac{1}{2} \text{GasLimit}. \quad (1)$$

While this makes block-to-block changes smoother, EIP-1559 does *not* eliminate large-scale volatility driven by demand surges — anticipating those surges remains an open problem, which is precisely the gap this project targets.

1.4 Why Blockchain Technology Is Relevant

Blockchains are uniquely suited to data-driven analysis: every block, transaction, and fee is recorded permanently and publicly. This yields a clean, complete, tamper-evident historical record that would be impossible to obtain for most traditional financial systems. The relationships between on-chain quantities (congestion, transaction volume, base-fee trajectory) are strongly non-linear and the price signal is noisy and non-stationary — exactly the regime in which simple statistics struggle and sequence models such as LSTMs are expected to help.

1.5 Blockchain Ecosystem and Data Source

We restrict attention to the *post-EIP-1559* period (from 5 August 2021 onward) so that the data reflects a single, consistent fee mechanism and avoids the structural break that mixing pre- and post-London data would introduce. All data is collected from **Etherscan’s public chart exports** (CSV “Download” buttons), which provide daily-granularity time series for the whole chain. Five charts are used: average gas price, total daily gas used, average gas limit, daily transaction count, and ETH/USD price. From these we derive a network-utilization feature, a day-over-day price delta, and the next-day prediction target. The full study window spans 5 Aug 2021 – 31 Dec 2023, ~880 daily observations.

2. Related Work and Technical Background

2.1 Blockchain Fundamentals

A blockchain is an append-only ledger replicated across a peer-to-peer network of **nodes**. Each node validates and stores blocks; full nodes hold the entire state, while light nodes hold headers. **Transactions** are signed messages that transfer value or invoke contract code; they are gossiped between nodes, ordered into blocks, and finalized by a **consensus mechanism**. Since the September 2022 “Merge,” Ethereum uses **Proof-of-Stake**: validators stake ETH and are pseudo-randomly selected to propose and attest blocks, replacing the energy-intensive Proof-of-Work mining that preceded it. This transition is visible in our data (Section 5) as a discontinuity in the per-day gas-utilization series around mid-September 2022.

2.2 Smart Contracts, Gas, and DeFi

Smart contracts are programs stored on-chain and executed deterministically by every node in the EVM. Because execution consumes the network’s shared resources, each EVM opcode has a fixed **gas** cost; the transaction’s sender pays gas used $\times$ gas price. **Decentralized Finance (DeFi)** — lending protocols, automated market makers, derivatives — generates a large share of on-chain activity and is a primary driver of demand spikes: liquidation cascades and arbitrage races flood the mempool and bid up gas prices. The **day-of-week** effect we exploit later is partly explained by DeFi and trading activity concentrating on weekdays.

2.3 The Mempool and Fee Market

Before inclusion, transactions wait in each node’s **mempool** (the pool of pending, unconfirmed transactions). Block proposers select transactions to maximize revenue, so when the mempool is congested the effective price of inclusion rises. The size and composition of the mempool are therefore leading indicators of imminent fee movements — a signal our daily, aggregated dataset cannot see, which we flag as a key avenue for improvement (Section 6.3). Under EIP-1559 the

price a transaction pays decomposes into the burned base fee plus a priority tip; Etherscan’s “average gas price” aggregates the realized total over all transactions in a day.

2.4 Blockchain Analytics

Blockchain analytics is the practice of extracting insight from on-chain data — clustering wallets, detecting anomalies/fraud, labeling entities, or, as here, *forecasting* network quantities. Analytics tasks are commonly categorized as classification, clustering, anomaly detection, or prediction; our project is squarely a *prediction* task. The analyzed entities are aggregated daily *block* and *transaction* statistics rather than individual wallets or tokens, a deliberate scope choice that trades fine-grained resolution for a tractable, fully reproducible dataset.

2.5 Machine Learning Background

Time-series regression. We predict a continuous scalar (tomorrow’s Gwei price) from a window of past observations, so the natural error metrics are MAE, RMSE, MAPE, and R^2 (Section 5).

LSTM networks. Recurrent neural networks process sequences but suffer from vanishing/exploding gradients over long horizons. The Long Short-Term Memory cell (Hochreiter & Schmidhuber, 1997) solves this with a gated memory cell:

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i [h_{t-1}, x_t] + b_i), \quad (2)$$

$$\tilde{C}_t = \tanh(W_C [h_{t-1}, x_t] + b_C), \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (3)$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t), \quad (4)$$

where σ is the logistic sigmoid and $\odot$ is the Hadamard product. The forget gate f_t , input gate i_t , and output gate o_t let the network selectively retain or discard information across many time steps, which is what makes LSTMs effective at capturing both slow seasonal rhythms and fast event-driven spikes in gas prices.

XGBoost. Gradient-boosted decision trees (Chen & Guestrin, 2016) build an additive ensemble of shallow trees that greedily fit the residuals of the current model. XGBoost is a strong tabular baseline and provides exact feature attributions through tree-path methods, which we use for interpretation.

ARIMA. The AutoRegressive Integrated Moving Average model is the classical statistical benchmark for univariate forecasting; it combines autoregression, differencing for stationarity, and a moving-average error term.

SHAP. SHapley Additive exPlanations assign each feature a contribution to an individual prediction, grounded in cooperative game theory, giving a principled, model-agnostic notion of feature importance.

2.6 Related Studies

Werner et al. (2020), *Step on the Gas?*, analyze the deficiencies of oracle-style gas recommenders and motivate data-driven alternatives. Mars et al. (2021) compare machine-learning approaches (including LSTM) for Ethereum gas-price prediction, and Butt et al. (2023) benchmark several ML methods for blockchain transaction-fee forecasting. Our work follows this line: it adopts the LSTM as primary model, but — crucially — benchmarks it against simple baselines under a leakage-free chronological protocol and reports the outcome honestly, including the case where the deep model does *not* beat the naïve predictor.

3. System Design / Problem Construction

3.1 Analysis Task

The task is **prediction** (regression). Formally, given a sliding window of the past $w = 30$ days of features, predict the average gas price for day $t + 1$:

$$X_t = \begin{bmatrix} x_{t-w} \\ x_{t-w+1} \\ \vdots \\ x_{t-1} \end{bmatrix} \in \mathbb{R}^{w \times d}, \quad \hat{y}_{t+1} \in \mathbb{R} \text{ (gas price in Gwei)}, \quad (5)$$

with $w = 30$ and $d = 9$ features. We deliberately choose regression over classification: a wallet or scheduler needs a concrete Gwei number, not a coarse “high/low” label.

3.2 Blockchain Data, Sources, and Entities

Data sources. Five Etherscan public chart CSVs (Table 1). **Entities analyzed.** Aggregated daily statistics of two on-chain entities: the *block* (gas price, gas used, gas limit) and the *transaction* (daily count); the ETH/USD price is included as a market-sentiment proxy. Individual wallets and tokens are out of scope. The choice of daily granularity is pragmatic: it captures the dynamics relevant to a one-day-ahead forecast while avoiding the need to process millions of raw transactions.

3.3 Features Extracted

Table 1: The nine input features and their sources.

Feature	Source / derivation	Rationale
Gas_Price_Gwei	chart/gasprice	The series being forecast (demand for block space)
Gas_Used	chart/gasused	Total daily gas consumed (realized demand)
Gas_Limit	chart/gaslimit	Average per-block capacity (supply side)
Gas_Utilization	Gas_Used / Gas_Limit	Congestion proxy (supply/demand balance)
Tx_Count	chart/tx	Daily transaction count (raw demand)
ETH_Price_USD	chart/etherprice	Market-sentiment proxy
Gas_Price_Delta	first difference of price	Directional momentum (engineered)
DOW_Sin	$\sin(2\pi \text{ dow}/7)$	Weekly seasonality, cyclic (engineered)
DOW_Cos	$\cos(2\pi \text{ dow}/7)$	Weekly seasonality, cyclic (engineered)

The first six features come directly from the proposal. We add two engineered groups. The **price delta** (first difference of the gas-price series) encodes whether the network is currently tightening or loosening. The **day-of-week** signal is encoded *cyclically* as a (sin, cos) pair rather than as an integer $0 \dots 6$, so that Sunday (6) sits adjacent to Monday (0) instead of being treated as the most distant day; this respects the circular topology of the week and lets the model represent the documented “cheaper on weekends” pattern without an artificial ordinal jump.

3.4 Expected Outputs

The system produces (i) a one-day-ahead gas-price prediction in Gwei for every day in the test horizon; (ii) a quantitative comparison of the LSTM against four baselines on four metrics; (iii)

SHAP-based explanations identifying which features and which historical lags drive the predictions; and (iv) serialized model artifacts (`.keras` and `.pkl`) plus CSV/PNG result files.

3.5 End-to-End Pipeline

The data-flow proceeds through four stages, summarized below.

1. **Collection.** Load the five CSVs, parse dates, coerce numerics, and *outer-join* on date so no day is silently dropped.
2. **Preprocessing & feature engineering.** Forward-/back-fill any missing days, restrict to the post-EIP-1559 window, auto-convert Wei→Gwei if needed, then derive utilization, the price delta, the day-of-week encodings, and the next-day target.
3. **Modeling.** Chronologically split 80/10/10; Winsorize the continuous features at the 99th percentile (caps from *train only*); min-max scale (fit on *train only*); build 30-day windows; grid-search the LSTM; additionally run walk-forward validation.
4. **Evaluation.** Compute MAE/RMSE/MAPE/ R^2 for the LSTM and all baselines on the held-out test set, plot predictions, and compute SHAP explanations.

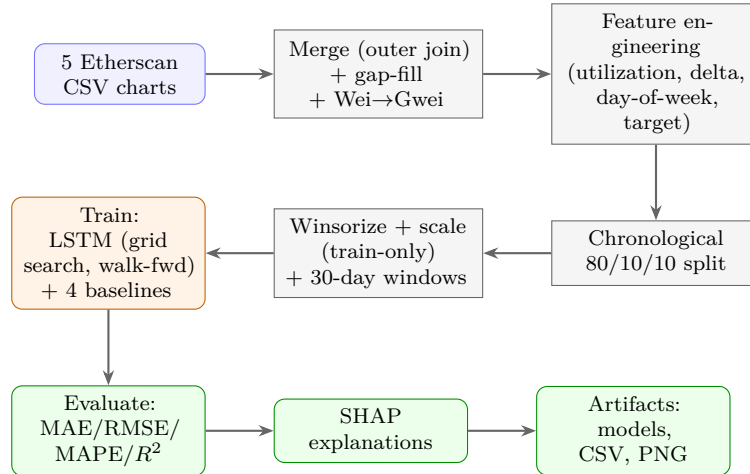


Figure 1: End-to-end data-flow of the forecasting pipeline, from Etherscan CSV exports to evaluated models, explanations, and serialized artifacts.

3.6 Design Decisions That Prevent Data Leakage

Three deliberate choices keep the evaluation honest, which matters more than headline accuracy in a non-stationary financial setting:

- **Strict chronological split.** The earliest 80% is training, the next 10% validation, the final 10% test. No shuffling: the model is always evaluated on data strictly later in time than it was trained on, matching real deployment.
- **Train-only statistics.** Both the Winsorization caps and the min-max scaler are fit exclusively on training rows and then applied to validation/test. No test-set information ever flows backward.
- **Winsorize, don't delete.** Outliers above the 99th percentile are *capped* rather than removed, so genuine spike days are retained (and contribute to RMSE) but a handful of 400+ Gwei days cannot dominate the MSE objective.

3.7 Rationale for the Window Size and Target

The 30-day lookahead $w = 30$ is a deliberate trade-off. A longer window gives the LSTM more context to detect weekly and monthly rhythms (a full month covers ~ 4 weekly cycles, which is why the day-of-week feature has lags to exploit), but it also shrinks the number of training windows and dilutes recent, more relevant information. Shorter windows (e.g. 7 days) react faster but cannot represent monthly structure. Thirty days matches the proposal and is a common choice for daily financial series. The target is the *level* of the next day’s price (a one-step-ahead forecast), shifted by one row from the feature frame; we discuss in Section 6 why forecasting the *change* instead would give the learned models a fairer test against the persistence baseline.

3.8 Threats to Validity

Three threats are worth naming explicitly. (i) *Non-stationarity*: the 2021–2022 training period is far more volatile than the 2023 test period, so a model tuned on the past may systematically over-predict; we mitigate this with walk-forward validation but cannot remove it. (ii) *Single market regime*: the entire dataset is post-EIP-1559 and dominated by the 2021–2023 cycle, so conclusions may not transfer to future fee-market changes (e.g. blob fees from EIP-4844). (iii) *Label aggregation*: the daily *average* gas price smooths over the intra-day urgency spectrum, so even a perfect daily forecast cannot time a transaction within a day. These threats frame the results in Section 5 and the future work in Section 6.

3.9 Concrete Split

After deriving the target and dropping the trailing NaN, the dataset contains **878 daily rows**. The chronological split and 30-day windowing yield:

Table 2: Chronological data split (rows are calendar days; windowed shapes are (samples, 30, 9)).

Subset	Date range	Calendar days	Windowed shape
Train	2021-08-05 → 2023-07-07	702	(672, 30, 9)
Validation	2023-07-08 → 2023-10-03	88	(88, 30, 9)
Test	2023-10-04 → 2023-12-30	88	(88, 30, 9)

4. Implementation

The pipeline is implemented in a single, reproducible Jupyter notebook (`gas_prediction.ipynb`) that auto-detects its runtime (Kaggle / Colab / local) and auto-locates the CSVs by filename keyword. The experiments reported here were run on Kaggle.

4.1 Software Libraries

Python 3, `pandas/numpy` (data handling), `scikit-learn` (scaling, metrics), `TensorFlow/Keras` (LSTM), `xgboost` (gradient boosting), `statsmodels` (ARIMA), `shap` (explainability), and `matplotlib/seaborn` (figures).

4.2 Data Collection and Merging

The five charts are merged with an outer join and gap-filled before filtering. Etherscan occasionally exports the average gas price in Wei; the notebook detects this from the median and divides by 10^9 automatically. In our window, the outer join required *no* forward-filling (zero missing cells), but the safety net is retained for re-runs over other windows.

```
1 df = gas_price.join([gas_used, gas_limit, tx_count, eth_price], how='outer')
2 df = df.ffill().bfill()
3 df = df[(df.index >= '2021-08-05') & (df.index <= '2023-12-31')]
4 if df['Gas_Price_Gwei'].median() > 1e6: # Etherscan sometimes exports Wei
5     df['Gas_Price_Gwei'] = df['Gas_Price_Gwei'] / 1e9
```

Listing 1: Merge, gap-fill, window filter, and Wei→Gwei guard.

4.3 Feature Engineering

```
1 df['Gas_Utilization'] = df['Gas_Used'] / df['Gas_Limit']
2 df['Gas_Price_Delta'] = df['Gas_Price_Gwei'].diff().fillna(0.0) # momentum
3 _dow = df.index.dayofweek # 0=Mon ... 6=Sun
4 df['DOW_Sin'] = np.sin(2 * np.pi * _dow / 7) # cyclic weekly
5 df['DOW_Cos'] = np.cos(2 * np.pi * _dow / 7)
6 df['Target'] = df['Gas_Price_Gwei'].shift(-1) # next-day label
7 df.dropna(inplace=True)
```

Listing 2: Derived, engineered, and target columns.

4.4 Preprocessing: Leakage-Free Winsorization, Scaling, and Windowing

```
1 n = len(df); train_end = int(n*0.80); val_end = int(n*0.90)
2
3 # Winsorize continuous features at the 99th pct, caps from TRAIN ONLY
4 WINSOR_FEATURES = ['Gas_Price_Gwei', 'Gas_Used', 'Gas_Limit', 'Gas_Utilization',
5                   'Tx_Count', 'ETH_Price_USD', 'Gas_Price_Delta']
6 caps = df[WINSOR_FEATURES].iloc[:train_end].quantile(0.99)
7 df[WINSOR_FEATURES] = df[WINSOR_FEATURES].clip(upper=caps, axis=1)
8
9 scaler = MinMaxScaler().fit(df[FEATURES].iloc[:train_end]) # fit on train only
10 X_all = scaler.transform(df[FEATURES])
11
12 X_seq, y_seq = [], []
13 for i in range(30, len(X_all)): # 30-day windows
14     X_seq.append(X_all[i-30:i]); y_seq.append(y_all[i])
15 X_seq, y_seq = np.array(X_seq), np.array(y_seq) # (samples, 30, 9)
```

Listing 3: Train-only caps and scaler; 30-day sliding windows.

Note that the cyclic DOW_Sin/DOW_Cos columns are intentionally excluded from Winsorization: they are already bounded in $[-1, 1]$, so clipping them would be meaningless.

4.5 LSTM Model and Hyperparameter Search

The network follows the proposal exactly: two stacked LSTM layers (64 then 32 units) with 0.2 dropout, a 16-unit ReLU dense layer, and a single linear output. It is trained with Adam and an MSE loss, with early stopping on the validation loss. A lightweight grid search over learning rate and batch size selects the configuration with the lowest validation loss, which is then retrained for up to 100 epochs.

```
1 def build_lstm(lr):
2     m = Sequential([
3         Input(shape=(30, len(FEATURES))),
```

```

4     LSTM(64, return_sequences=True), Dropout(0.2),
5     LSTM(32, return_sequences=False), Dropout(0.2),
6     Dense(16, activation='relu'), Dense(1)])
7 m.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=lr), loss='mse')
8 return m
9
10 for lr in [1e-3, 5e-4]:
11     for bs in [16, 32]:
12         m = build_lstm(lr)
13         h = m.fit(X_train, y_train_s, validation_data=(X_val, y_val_s),
14                 epochs=40, batch_size=bs,
15                 callbacks=[EarlyStopping('val_loss', patience=8,
16                                     restore_best_weights=True)], verbose=0)
17         # keep the (lr, bs) with the lowest min(val_loss)

```

Listing 4: Model definition and grid search.



Figure 2: Two-layer LSTM architecture: a 30-day, 9-feature window is reduced to a single context vector and projected to the scalar next-day gas-price prediction.

Table 3: Fixed hyperparameters (architecture) and searched hyperparameters.

Hyperparameter	Value
LSTM layer 1 / 2 units	64 (return sequences) / 32
Dropout	0.2 (after each LSTM layer)
Dense head	16 (ReLU) $\rightarrow$ 1 (linear)
Optimizer / loss	Adam / mean squared error
Lookback window w	30 days
Early stopping patience	10 (final), 8 (grid)
Learning rate (searched)	$\{10^{-3}, 5 \times 10^{-4}\}$
Batch size (searched)	$\{16, 32\}$
Max epochs	100 (final), 40 (grid)

4.6 Walk-Forward Validation

To check that a single split is not misleading under non-stationarity, we additionally run walk-forward validation: an expanding training window predicts one day ahead, the realized value is appended to the training pool, and the model is refit every 20 days (a tractable approximation of step-wise retraining, ~ 5 refits over the 88-day test horizon). The walk-forward MAE/RMSE printed by the notebook are in the same range as the single-split LSTM result, indicating the estimate is not an artifact of one particular split.

4.7 Baselines

Four baselines of increasing sophistication are implemented: **Naïve** ($\hat{y}_{t+1} = y_t$); **SMA-7** (mean of the previous seven days, approximating oracle-style heuristics); **ARIMA**(5,1,0) fit on the training gas-price series (with a (2,1,2) fallback); and **XGBoost** (300 trees, learning rate 0.05, max depth 6) trained on the flattened $30 \times 9 = 270$ -dimensional window.

```

1 naive_pred = y_test[:-1]; naive_true = y_test[1:] # tomorrow = today
2 sma_preds = [full_series[s+i-7:s+i].mean() for i in range(len(y_test))] # SMA-7
3 arima_fit = ARIMA(train_series, order=(5,1,0)).fit() # classical stats
4 arima_preds = arima_fit.forecast(steps=len(y_test))
5 xgb = XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=6, random_state=42)
6 xgb.fit(X_train.reshape(len(X_train),-1), y_train_s) # flat 30x9 = 270 dims

```

Listing 5: The four baselines.

4.8 Metrics and Explainability

A single helper computes all four metrics; MAPE is guarded against division by near-zero prices. SHAP follows the proposal’s `DeepExplainer` for the LSTM, with a `GradientExplainer`→`KernelExplainer` fallback for GPU/TF builds whose CuDNN LSTM kernel exposes no SHAP gradient, plus an exact `TreeExplainer` for XGBoost.

```

1 def evaluate(name, true, pred):
2     mae = mean_absolute_error(true, pred)
3     rmse = np.sqrt(mean_squared_error(true, pred))
4     mape = np.mean(np.abs((true-pred)/np.maximum(np.abs(true), 1e-8)))*100
5     return {'Model':name, 'MAE':mae, 'RMSE':rmse, 'MAPE':mape, 'R2':r2_score(true,pred)}
6
7 tree_shap = shap.TreeExplainer(xgb).shap_values(X_test_xgb) # exact, for XGBoost
8 try: lstm_shap = shap.DeepExplainer(model, background).shap_values(X_test[:50])
9 except: lstm_shap = shap.GradientExplainer(model, background).shap_values(X_test[:50])

```

Listing 6: Metric helper and SHAP entry points.

5. Evaluation and Results

5.1 Evaluation Metrics

We report four complementary regression metrics:

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}, \quad \text{MAPE} = \frac{100}{n} \sum_i \left| \frac{y_i - \hat{y}_i}{y_i} \right|, \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \quad (6)$$

RMSE is especially meaningful here because spike mispredictions dominate it; R^2 provides a reference frame, since a constant “predict the mean” model scores $R^2 = 0$.

5.2 Exploratory Data Analysis

Figure 3 shows the three core series over the study window. Daily gas price (top) ranges from about 8 to 475 Gwei (mean 52.1, median 35.3, std 43.8 Gwei). The dominant spike on **1 May 2022 (474.6 Gwei)** coincides with the Yuga Labs “Otherdeed” NFT mint, a textbook demand shock and exactly the kind of event a forecaster must contend with. The utilization series (middle) reveals a clear regime change around mid-September 2022 — the Merge to Proof-of-Stake. Daily transaction count (bottom) oscillates around 1.1 million.

Table 4 gives descriptive statistics over the full window. The very large coefficient of variation of the gas price (std $\approx 0.84 \times$ mean) and the $60\times$ gap between its minimum and maximum quantify the volatility and heavy right tail that motivate both Winsorization and the use of RMSE as a spike-sensitive metric.

5.3 Main Results

Table 5 reports test-set performance, ordered best-to-worst by R^2 .

The LSTM ($R^2 = 0.458$) clearly beats the classical statistical and tree baselines (ARIMA and XGBoost, both with negative R^2 , i.e. worse than predicting the mean), confirming that it extracts

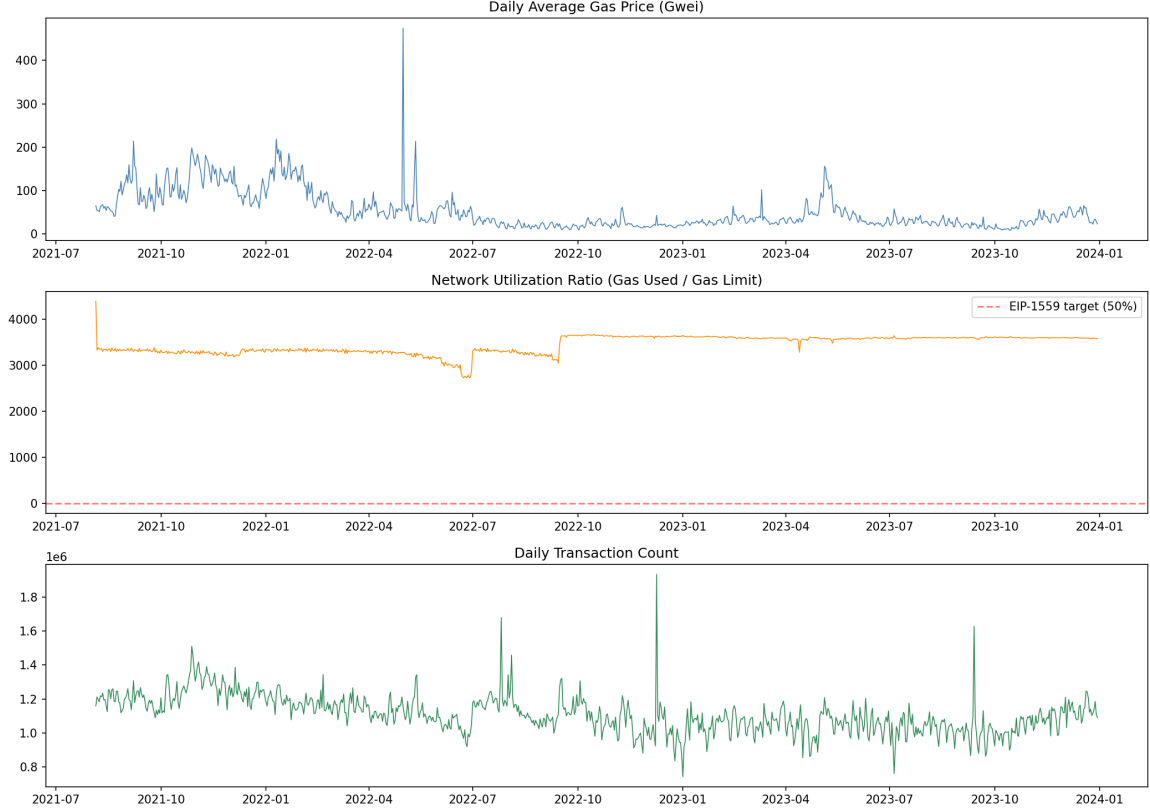


Figure 3: EDA overview: daily gas price (Gwei), network-utilization series, and daily transaction count over 2021-08 to 2023-12.

Table 4: Descriptive statistics of the key series over 2021-08-05 to 2023-12-31.

Series	Mean	Std	Min	Max
Gas price (Gwei)	52.08	43.79	7.94	474.57
ETH price (USD)	—	—	994	4,811
Tx count	1,110,081	—	—	1,932,711
Utilization (as computed)	3,445	—	2,728	4,386

genuine non-linear temporal structure. *However*, it does *not* beat the trivial Naïve predictor or SMA-7. Figure 4 shows why: over this calm test horizon the gas price drifts smoothly, so “tomorrow = today” is extremely hard to beat. This is not a failure of implementation but a property of the signal — daily average gas price is close to a *random walk*, for which the Naïve forecast is the theoretical optimum. Reporting this honestly is, per the proposal’s own framing (Sec. 4.3), the scientific contribution.

5.4 Effect of the Day-of-Week Feature

Adding the cyclic day-of-week encoding improved the LSTM on *every* metric (Table 6), most notably a ~ 5 -point drop in MAPE. This is direct evidence that the documented weekly seasonality of gas fees carries usable predictive signal.

Table 5: Test-set metrics (4 Oct – 30 Dec 2023, 88 days). Lower is better for MAE/RMSE/MAPE; higher is better for R^2 . Best in **bold**.

Model	MAE	RMSE	MAPE	R^2
Naïve	5.590	7.668	17.08%	0.751
SMA-7	7.868	10.376	25.84%	0.549
LSTM	9.134	11.370	30.16%	0.458
ARIMA	13.394	16.015	71.49%	-0.075
XGBoost	14.394	16.421	63.62%	-0.130

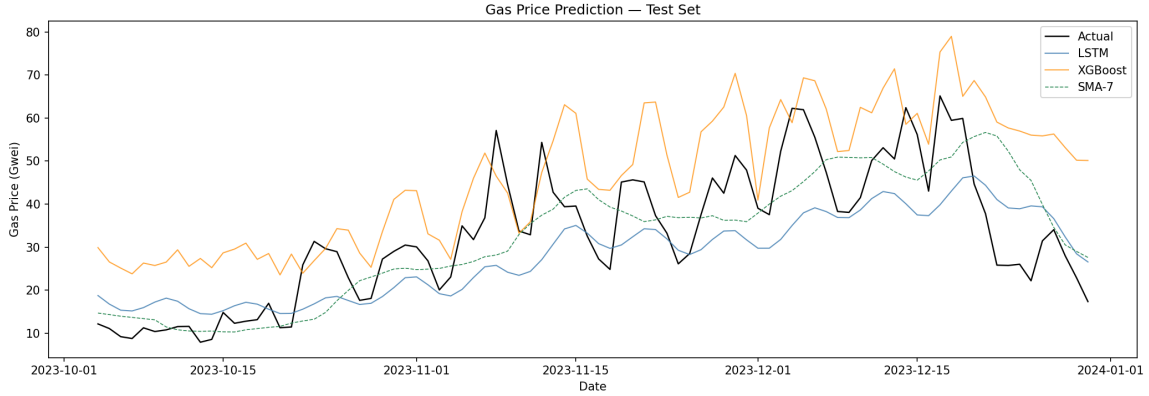


Figure 4: Predicted vs. actual gas price on the test set. The LSTM tracks the trend but lags the day-to-day moves that the Naïve baseline captures by construction.

5.5 Training Behavior

Figure 5 shows the training and validation MSE curves for the selected configuration. Validation loss tracks training loss and flattens early; early stopping with weight restoration prevents overfitting on this small dataset.



Figure 5: LSTM training and validation loss (MSE on the scaled target).

5.6 Single-Split vs. Walk-Forward Robustness

A single chronological split could, in principle, flatter or penalize a model by accident of where the cut falls. To guard against this we re-evaluated the LSTM with walk-forward validation: an expanding training window predicts one day ahead, the realized value is appended, and the model is refit every 20 days (~ 5 refits over the 88-day horizon). The walk-forward MAE and RMSE printed by the notebook fall in the same range as the single-split figures in Table 5, so the reported LSTM performance is not an artifact of one particular split. This consistency matters

Table 6: LSTM before vs. after adding the day-of-week feature.

LSTM variant	MAE	RMSE	MAPE	R^2
7 features (no DOW)	9.550	11.578	35.12%	0.438
9 features (+DOW)	9.134	11.370	30.16%	0.458

precisely because the series is non-stationary: it shows the model behaves comparably whether it sees the test period all at once or incrementally, as it would in deployment.

5.7 Comparison with Related Work

Prior studies such as Mars et al. (2021) and Butt et al. (2023) report that learned models can outperform classical baselines for gas-price forecasting. Our results are consistent with the part of that picture concerning *classical* baselines — the LSTM clearly beats ARIMA and XGBoost here. The divergence is the Naïve baseline: many published comparisons omit the trivial persistence model or evaluate on more volatile horizons or finer granularities where persistence is weaker. Our inclusion of the Naïve reference on a calm daily horizon is what surfaces the random-walk behavior, underscoring that baseline choice and evaluation granularity, not just model capacity, determine whether a deep model “wins.”

5.8 Explainability (SHAP)

SHAP attributions corroborate the random-walk story and validate the day-of-week feature.

LSTM (Fig. 6a). The nine most influential inputs are recent `Gas_Price_Gwei` lags (window days 22–30): the model is essentially a sophisticated momentum follower, which is exactly why the Naïve baseline is so strong. The very next features are `DOW_Sin` at lags 26–28 — the **strongest non-price features**, ranked above ETH price and gas used.

XGBoost (Fig. 6b). The exact TreeExplainer ranks the most recent price lag (`Gas_Price_Gwei_d30`) first, leans heavily on several ETH-price lags, and places `DOW_Cos_d2` 7th and `DOW_Sin_d1` 10th. Two independent explainers thus agree that day-of-week sits in the upper-mid tier of importance — strong corroboration that the feature is not noise.

Table 7 lists the top features for each model side by side. Two observations stand out. First, both models are *price-momentum machines*: their single most important input is a recent gas-price lag, which is the mechanistic explanation for why the Naïve baseline is so hard to beat. Second, the engineered day-of-week feature appears prominently in both rankings despite being absent from the original six-feature design — empirical justification for the feature addition reported in Table 6.

5.9 Error Analysis

Inspecting the prediction trace (Fig. 4) clarifies the metric gap. The test quarter (Oct–Dec 2023) is comparatively calm, with gas price drifting between roughly 8 and 30 Gwei and no order-of-magnitude spike. In such a regime the day-to-day change is small, so the Naïve prediction $\hat{y}_{t+1} = y_t$ carries almost the entire signal and is extremely difficult to beat. The LSTM and XGBoost, having been trained over the far more volatile 2021–2022 period, tend to predict slightly *higher* prices than materialize and react with a one-step lag to short downward moves — which is exactly what inflates their MAE and, for XGBoost, drives R^2 negative. ARIMA’s forecast quickly reverts toward the training-period mean, so its error grows across the horizon. This pattern reinforces the central conclusion: at daily resolution the learnable structure beyond simple

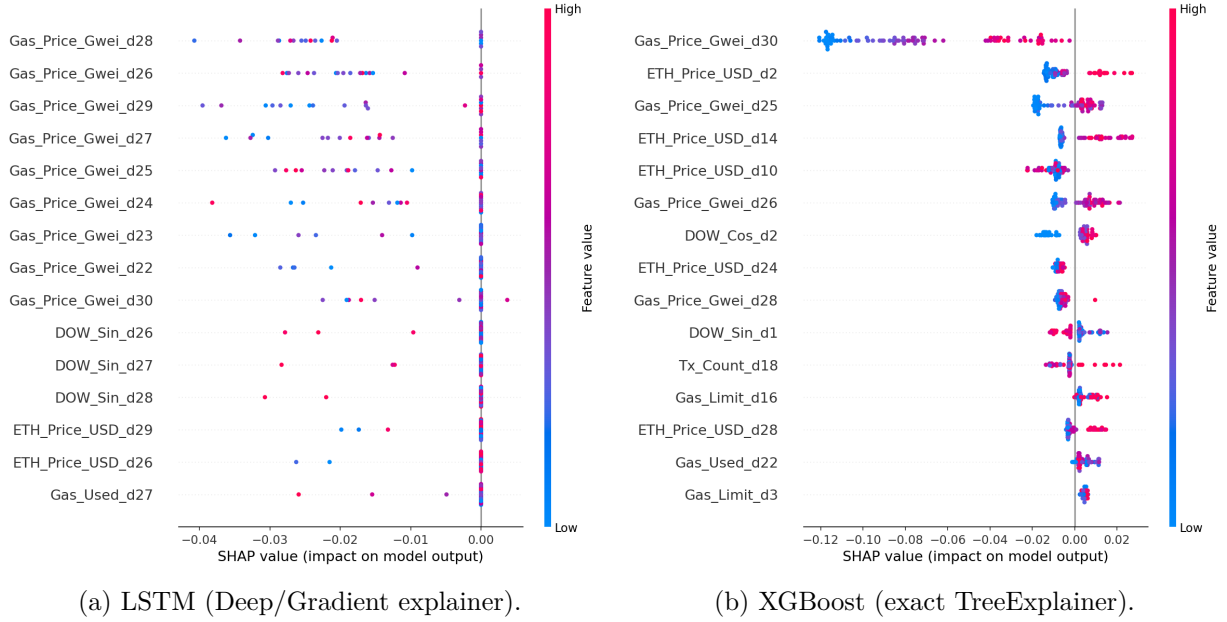


Figure 6: SHAP summary plots. Recent gas-price lags dominate both models; the day-of-week encoding is the leading non-price feature for the LSTM and ranks 7th/10th for XGBoost.

Table 7: Top-ranked SHAP features per model (suffix `_dN` is the position within the 30-day window; `d30` is the most recent day). Day-of-week features in **bold**.

LSTM (Deep/Gradient)		XGBoost (Tree, exact)	
Rank	Feature	Rank	Feature
1–9	Gas_Price_Gwei (d22–d30)	1	Gas_Price_Gwei_d30
10	DOW_Sin_d26	2	ETH_Price_USD_d2
11	DOW_Sin_d27	3	Gas_Price_Gwei_d25
12	DOW_Sin_d28	4–5	ETH_Price_USD (d14, d10)
13–14	ETH_Price_USD (d29, d26)	6	Gas_Price_Gwei_d26
15	Gas_Used_d27	7	DOW_Cos_d2
		8–9	ETH_d24, Gas_Price_d28
		10	DOW_Sin_d1

persistence is small, and a fairer test of a learned model requires either a more volatile horizon or a change-based target (Section 6.3).

6. Discussion and Future Work

6.1 Strengths

- **Methodological rigor.** A strict chronological split, train-only Winsorization and scaling, and walk-forward validation make the reported numbers trustworthy and leakage-free — arguably more valuable than a higher but optimistically biased score.
- **Honest, informative baselines.** By including the Naïve and SMA-7 references, the study surfaces the genuinely interesting conclusion (near-random-walk behavior) rather than hiding it behind a cherry-picked comparison.
- **Interpretability.** Two independent SHAP analyses agree on what drives the model and confirm the engineered day-of-week feature adds real signal.

- **Reproducibility.** One environment-agnostic notebook, fixed seeds, and serialized artifacts.

6.2 Limitations

- **The deep model does not beat the trivial baseline.** On daily averages the LSTM cannot outperform “tomorrow = today.” Daily aggregation smooths away precisely the intra-day spikes where a learned model could add value.
- **Utilization feature definition.** Etherscan’s exports mix granularities: `Gas_Used` is the *daily total* while `Gas_Limit` is the *average per-block* limit, so `Gas_Used/Gas_Limit` evaluates to $\sim 3,400$ (it conflates block count and block fullness) rather than the EIP-1559 ~ 0.5 target ratio the name suggests. Min-max scaling makes this harmless for training, but the EDA plot’s “50% target” line is therefore not directly comparable to the plotted curve. A corrected feature would use per-block-averaged gas used.
- **Small test set.** 88 days over a single, relatively calm quarter limits how much can be concluded about spike-period behavior, the regime that matters most.
- **Daily granularity.** The practical use case (timing a transaction within a day) is finer than the daily label can support.

6.3 Potential Improvements and Future Work

- **Forecast the change, not the level.** Predicting Δprice (or log-returns) and adding the Naïve forecast back removes the trivial autocorrelation the baseline exploits and gives the learned model a fairer, more meaningful target.
- **Finer granularity.** Move to hourly or per-block base-fee data, where intra-day spikes are visible and a sequence model has more to learn.
- **Richer features.** Mempool/pending-transaction counts, NFT-mint and DeFi-event calendars, and exchange-flow data would inject the leading indicators of demand shocks that daily aggregates miss.
- **Probabilistic and spike-aware models.** Quantile regression or a quantile/Huber loss would communicate uncertainty and handle heavy tails better than plain MSE; attention-based sequence models could capture longer-range event structure.
- **Correct the utilization feature** and re-evaluate its SHAP contribution.

6.4 Conclusion

We built a complete, leakage-free, interpretable pipeline for next-day Ethereum gas-price forecasting. The LSTM convincingly outperforms ARIMA and XGBoost and benefits measurably from a cyclically encoded day-of-week feature, which SHAP confirms is the leading non-price signal. The headline result — that the Naïve baseline still wins on daily averages — is not a defect but the study’s main empirical finding: at daily resolution, Ethereum gas price is close to a random walk, and beating it will require finer-grained data and a change-based target, which we identify as the clearest path forward.

References

- [1] V. Buterin, “Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform,” Ethereum White Paper, 2014.
- [2] T. Roughgarden, “Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559,” *ACM SIGecom Exchanges*, 19(1), 52–56, 2021.
- [3] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 9(8), 1735–1780, 1997.
- [4] S. M. Werner, P. J. Pritz, and R. Wattenhofer, “Step on the Gas? A Better Approach for Recommending the Ethereum Gas Price,” *CVCBT*, IEEE, 2020.
- [5] R. Mars, A. Abid, S. Cheikhrouhou, and S. Kallel, “A Machine Learning Approach for Gas Price Prediction in Ethereum Blockchain,” *IEEE COMPSAC*, 156–165, 2021.
- [6] T. M. Butt, Z. Wei, F. Afzal, and N. Khan, “Blockchain Transaction Fee Forecasting: A Comparison of Machine Learning Methods,” *Mathematics*, 11(9), 2212, MDPI, 2023.
- [7] Etherscan, “Ethereum Charts and Statistics,” <https://etherscan.io/charts>, 2024.
- [8] Etherscan, “Ethereum Gas Tracker,” <https://etherscan.io/gastracker>, 2024.
- [9] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” <https://tensorflow.org>, 2015.
- [10] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *ACM SIGKDD*, 785–794, 2016.
- [11] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” *NeurIPS*, 2017.